

101.3 & 102.3: Sistemas de inicialização & Controle de bibliotecas compartilhadas

Rafael Obelheiro

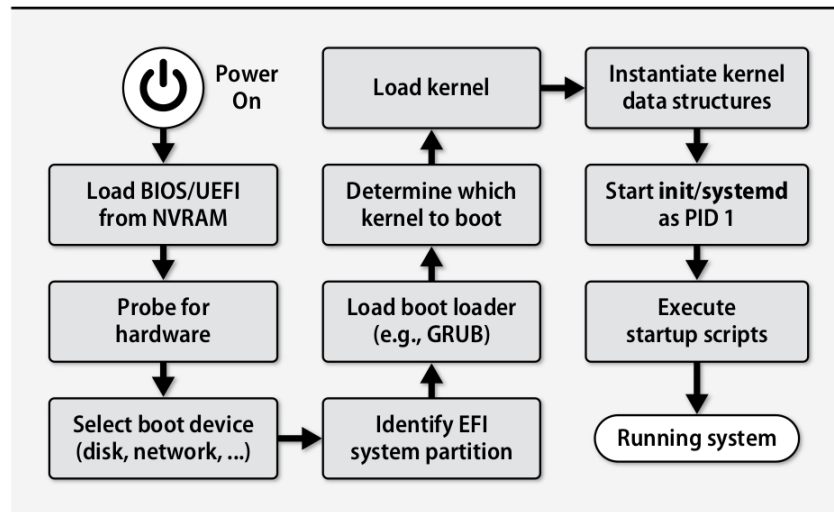
OTCC Linux

Section 1

101.3 Sistemas de inicialização

Processo de boot do Linux

Linux & UNIX boot process



Processos carregados no boot

- Após a carga e inicialização do kernel, vários processos são criados automaticamente
- Processos espontâneos
 - ▶ tarefas do kernel, muitas vezes sem executáveis associados
 - ▶ não requerem (nem permitem) administração
 - ▶ aparecem na lista de processos com PIDs baixos e nomes entre colchetes
 - ★ [kthreadd], [migration/1]
- Sistema de inicialização
 - ▶ responsável por configurar o sistema e iniciar serviços em espaço de usuário
 - ▶ raiz da árvore de processos, PID=1
 - ▶ diversas implementações
 - ★ SysV init, systemd, upstart, OpenRC

Funções de um sistema de inicialização

- O sistema de inicialização é responsável por tarefas como:
 - ▶ definir o nome da máquina
 - ▶ definir o fuso horário
 - ▶ verificar integridade e montar sistemas de arquivos
 - ▶ limpar áreas do sistema de arquivos (como /tmp)
 - ▶ configurar a rede
 - ▶ iniciar outros serviços
 - ▶ coordenar o desligamento/reboot do sistema
- Geralmente o sistema de inicialização invoca conjuntos de scripts para realizar essas tarefas

Variantes de sistemas de inicialização

- SysV init
 - ▶ (mais) simples e tradicional
 - ★ esquema básico remonta à década de 1980
 - ▶ semelhante ao usado em outras variantes de Unix (BSDs)
 - ▶ alguns conceitos e comandos subsistem em outras variantes
 - ★ runlevels, modo monousuário
- systemd
 - ▶ desenvolvido a partir de 2010, objetivo inicial era contornar limitações do SysV init
 - ★ execução puramente sequencial → boot demorado
 - ★ gerenciamento de dependências
 - ▶ cresceu em escopo e absorveu outros subsistemas
 - ▶ complexo e poderoso, específico do Linux
- upstart (Ubuntu, abandonado em favor do systemd)
- OpenRC (Gentoo)
- A maioria das principais distribuições usam systemd, mas ainda existem distribuições com SysV init (ou similar)

SysV init: níveis de execução (*runlevels*)

- O SysV init usa o conceito de nível de execução (*runlevel*) para definir o conjunto de serviços que devem ser disponibilizados
 - ▶ a tabela mostra um esquema típico, mas os runlevels 2–5 não são padronizados

runlevel	descrição
0	desliga o sistema
1, s, S	monousuário (manutenção)
2	multiusuário (sem serviços de rede)
3	multiusuário (com serviços de rede)
4	não padronizado
5	multiusuário com rede e GUI
6	reboot

SysV init: configuração

- A configuração do SysV init é dirigida pelo arquivo `/etc/inittab`
- Entradas no formato `id:runlevels:ação:processo`
 - ▶ `id`: identificador (até 4 caracteres)
 - ▶ `runlevels`: a quais runlevels a entrada se aplica
 - ★ ex: 235=runlevels 2, 3 e 5
 - ▶ `ação`: como `init` vai processar a entrada
 - ★ `sysvinit`, `boot`, `bootwait`: na inicialização (em ordem)
 - ★ `initdefault`: qual o runlevel default
 - ★ `wait`: executa nos runlevels especificados, espera o término para continuar
 - ★ `respawn`: reinicia o processo se ele for encerrado
 - ▶ `processo`: comando a ser executado

SysV init: exemplo de inittab

```
# Level to run in
id:5:initdefault:
# Boot-time system configuration/initialization script.
si::sysinit:/etc/init.d/rcS
# What to do in single-user mode.
~:S:wait:/sbin/sulogin
# /etc/init.d executes the S and K scripts upon change
# of runlevel.
l0:0:wait:/etc/init.d/rc 0
l1:1:wait:/etc/init.d/rc 1
l2:2:wait:/etc/init.d/rc 2
l3:3:wait:/etc/init.d/rc 3
l4:4:wait:/etc/init.d/rc 4
l5:5:wait:/etc/init.d/rc 5
l6:6:wait:/etc/init.d/rc 6
# What to do at the "3 finger salute".
ca::ctrlaltdel:/sbin/shutdown -t1 -h now
# Runlevel 2,3: getty on virtual consoles
1:23:respawn:/sbin/getty tty1 VC linux
2:23:respawn:/sbin/getty tty2 VC linux
3:23:respawn:/sbin/getty tty3 VC linux
```

SysV init: scripts

- O diretório `/etc/init.d` armazena os scripts de gerenciamento de serviços
- Cada runlevel `X` tem um diretório `/etc/rcX.d` com links simbólicos para os scripts em `/etc/init.d`
 - ▶ `SNNabc`: ao entrar no runlevel, invoca o script alvo com `start`
 - ▶ `KNNabc`: ao entrar no runlevel, invoca o script alvo com `stop`
 - ★ `NN` é um número de 2 dígitos que determina a sequência
 - ★ scripts são executados em ordem alfabética (`K*` antes de `S*`, ordem crescente de `NN`)
- Muitas distribuições têm um script `/etc/rc.local` ou `/etc/rc.d/rc.local` contendo comandos que são executados por último na inicialização

SysV init: scripts em `/etc/init.d`

- Os scripts em `/etc/init.d` aceitam parâmetros comuns
 - ▶ nem todos os scripts aceitam `reload`

parâmetro	significado
<code>start</code>	inicia o serviço
<code>stop</code>	encerra o serviço
<code>restart</code>	reinicia o serviço (=stop + start)
<code>reload</code>	relê arquivos de configuração
<code>status</code>	mostra estado do serviço

- O script `/etc/init.d/foo` também pode ser invocado com o comando `service foo`:

```
### inicia o serviço foo
$ sudo /etc/init.d/foo start
$ sudo service foo start
```

SysV init: comandos para runlevels

```
### mostram o runlevel atual
```

```
$ runlevel
```

```
N 5
```

```
$ who -r
```

```
run-level 5 1985-10-26 22:04
```

```
### muda o runlevel para 1
```

```
$ sudo telinit 1
```

```
### notifica init para reler /etc/inittab
```

```
$ sudo telinit q
```

systemd

- systemd organiza serviços e recursos em **unidades**
- Os principais tipos de unidades relacionadas a inicialização e serviços são
 - service: serviços
 - target (destino): agrupamentos de outras unidades
 - ★ neste contexto, similares a runlevels
- Cada unidade é definida e configurada por um arquivo
 - /lib/systemd/system: arquivos padrão
 - ★ não devem ser modificados
 - /etc/systemd/system: arquivos locais
 - ★ alguns pacotes criam arquivos aqui

systemctl

- systemctl é o comando usado para interagir com o systemd
- Os subcomandos mais usados são

Subcommand	Function
list-unit-files [<i>pattern</i>]	Shows installed units; optionally matching <i>pattern</i>
enable <i>unit</i>	Enables <i>unit</i> to activate at boot
disable <i>unit</i>	Prevents <i>unit</i> from activating at boot
isolate <i>target</i>	Changes operating mode to <i>target</i>
start <i>unit</i>	Activates <i>unit</i> immediately
stop <i>unit</i>	Deactivates <i>unit</i> immediately
restart <i>unit</i>	Restarts (or starts, if not running) <i>unit</i> immediately
status <i>unit</i>	Shows <i>unit's</i> status and recent log entries
kill <i>pattern</i>	Sends a signal to units matching <i>pattern</i>
reboot	Reboots the computer
daemon-reload	Reloads unit files and systemd configuration

Exemplos com systemctl

```
### Lista unidades ativas (todas, apenas serviços)
```

```
$ systemctl list-units
```

```
$ systemctl list-units --type=service
```

```
### Lista destinos instalados, ativos ou nao
```

```
$ systemctl list-unit-files --type=target
```

```
### Mostra status do serviço anacron
```

```
$ systemctl status anacron.service
```

```
### Ativa serviço argus
```

```
$ sudo systemctl start argus.service
```

```
### Inclui/remove ssh nos serviços ativados no boot
```

```
$ sudo systemctl enable ssh.service
```

```
$ sudo systemctl disable ssh.service
```

Estados de unidades

- O estado de uma unidade pode ser consultado com `systemctl status`
- A tabela abaixo mostra os estados mais frequentes
 - os mais relevantes do ponto de vista de administração são `enabled`, `disabled` e `masked`
 - unidades `disabled` podem ser ativadas manualmente, enquanto que unidades `masked` não podem ser ativadas

State	Meaning
bad	Some kind of problem within systemd ; usually a bad unit file
disabled	Present, but not configured to start autonomously
enabled	Installed and runnable; will start autonomously
indirect	Disabled, but has peers in Also clauses that may be enabled
linked	Unit file available through a symlink
masked	Banished from the systemd world from a logical perspective
static	Depended upon by another unit; has no install requirements

Destinos (*targets*)

- Os destinos (*targets*) funcionam como runlevels
- Os destinos mais usados são
 - `multi-user.target`: multiusuário, sem GUI
 - `graphical.target`: multiusuário, com GUI
 - `rescue.target`: monousuário (manutenção)
- O destino usado no boot pode ser mostrado com `systemctl get-default` e alterado com `systemctl set-default`
- O destino atual pode ser redefinido com `systemctl isolate`

```
### lista/altera o destino usado no boot
$ systemctl get-default
$ sudo systemctl set-default multi-user.target
```

```
### coloca o sistema em modo monousuário
$ sudo systemctl isolate rescue.target
```

Reiniciando e desligando o sistema

- A forma mais educada de desligar ou rebootar é usando `shutdown`
 - permite programar a ação para uma hora determinada
 - ★ 5 minutos antes do horário previsto, o arquivo `/etc/passwd` é criado, impedindo novos logins de usuários regulares
 - ★ ação programada pode ser cancelada
 - permite notificar os usuários logados
 - ★ mesmo efeito de `wall(1)`
- Se não houver necessidade de ser educado...
- quando não houver usuários a notificar e/ou a ação for imediata
- ...pode-se usar os comandos “de baixo nível”
 - `reboot`, `halt`, `poweroff`
 - ★ com `systemd`, `systemctl reboot|halt|poweroff` também funcionam

Exemplos (1)

```
### Reinicia em 1 minuto
$ sudo shutdown -r
```

```
### Desliga em 10 minutos
$ sudo shutdown -h +10
```

```
### Desliga às 17:30, notificando usuarios
$ sudo shutdown -h 17:30 Parada de fim de tarde
```

```
### Cancela todos os shutdowns pendentes
$ sudo shutdown -c
```

Exemplos (2)

```
### Desliga imediatamente (comandos equivalentes)
$ sudo shutdown -h now
$ sudo shutdown -P +0
$ sudo poweroff
$ sudo halt -p
```

```
### Para o sistema, sem desligar (comandos equivalentes)
$ sudo shutdown -H now
$ sudo halt
```

```
### Reinicia o sistema (comandos equivalentes)
### Ctrl-Alt-Del na console geralmente funciona, também
$ sudo shutdown -r now
$ sudo reboot
```

Section 2

103.2 Bibliotecas compartilhadas

Bibliotecas compartilhadas

- Programas executáveis precisam ter acesso a funções de biblioteca
 - `printf()`, `fgets()`, `qsort()`, ...
- Providenciar acesso às bibliotecas é tarefa do ligador (*linker*)
 - ligação estática: o executável incorpora o código e os dados necessários das bibliotecas
 - ligação dinâmica: o executável contém ponteiros para as bibliotecas que precisa, que são resolvidos durante a carga
 - ★ bibliotecas compartilhadas no Unix, *dynamic-link libraries* (DLLs) no Windows
- Ligação estática consome mais espaço em disco e na memória, e exige recompilar/religar os executáveis em caso de alterações na biblioteca

Nomeação de bibliotecas compartilhadas

- O nome de uma biblioteca compartilhada (*soname*) é geralmente formado por
 - nome da biblioteca (muitas vezes precedido por `lib`)
 - `so` (de *shared object*)
 - número de versão
 - ★ ex: `libc.so.6`, `libreadline.so.7`
 - é comum que o arquivo da biblioteca seja uma ligação simbólica para uma versão mais específica
 - ★ ex: `libc.so.6` -> `libc-2.27.so`
- Bibliotecas estáticas usam a extensão `.a`
 - ex: `libc.a`

Diretórios de bibliotecas compartilhadas

- Os diretórios são configurados via `/etc/ld.so.conf`
 - a diretiva `include` permite incluir outros arquivos
- Diretórios usuais incluem
 - `/lib`
 - `/lib32`
 - `/lib64`
 - `/usr/lib`
 - `/usr/local/lib`

ldconfig, LD_LIBRARY_PATH, ldd (1)

- O comando `ldconfig` atualiza o cache de biblioteca em `/etc/ld.so.cache`
 - deve ser executado sempre que a lista de diretórios for modificada
- A variável de ambiente `LD_LIBRARY_PATH` permite definir um conjunto de caminhos para busca de bibliotecas compartilhadas em uma sessão do shell
- O comando `ldd` mostra as bibliotecas compartilhadas com as quais um executável foi ligado

ldconfig, LD_LIBRARY_PATH, ldd (2)

```
### Atualiza o cache, mostrando as bibliotecas encontradas  
$ sudo ldconfig -v
```

```
### Mostra o conteúdo do cache  
$ ldconfig -p
```

```
### Busca temporariamente bibliotecas em ~/lib  
$ LD_LIBRARY_PATH=~/lib
```

```
### Lista as bibliotecas usadas por /bin/ls  
$ ldd /bin/ls
```